Capacitance PEX Benchmark: kpex vs MAGIC

Comparative analysis of parasitic capacitance extraction between:

- **klayout-pex (kpex)** — KLayout-based 2.5D extraction
- **MAGIC** — VLSI layout tool extraction (reference)

Organized by capacitance type:

1. **Substrate** — wire-to-substrate capacitance
2. **Overlap** — parallel-plate capacitance between vertically stacked metal layers
3. **Sidewall** — lateral coupling between adjacent conductors on the same layer
4. **Fringe / Side-Overlap** — fringe-field coupling between staggered conductors on adjacent layers
5. **Total / Cross Structures** — full coupling across all mechanisms

In [1]:
```python
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scienceplots  # noqa: F401
import seaborn as sns

plt.style.use(["science", "ieee", "no-latex"])
sns.set_palette("colorblind")

# Set to True to exclude substrate (VSUBS) capacitance from coupling comp
EXCLUDE_SUBSTRATE = True
```

Load Data

In [2]:
```python
PEX_DIR = Path("dataset/capacitance/pex")

TOOLS = {
    "kpex": {
        "cap": "kpex_capacitance.json",
        "time": "capacitance_timing_kpex.json",
        "mem": "capacitance_memory_kpex.json",
    },
    "magic": {
        "cap": "magic_capacitance.json",
        "time": "capacitance_timing_magic.json",
        "mem": "capacitance_memory_magic.json",
    },
}


def load_json(name):
```

```python
    p = PEX_DIR / name
    return json.loads(p.read_text()) if p.exists() else {}


cap_data = {t: load_json(v["cap"]) for t, v in TOOLS.items()}
time_data = {t: load_json(v["time"]) for t, v in TOOLS.items()}
mem_data = {t: load_json(v["mem"]) for t, v in TOOLS.items()}

print("Cells per tool (capacitance):")
for t, d in cap_data.items():
    print(f"  {t}: {len(d)}")
```

```
Cells per tool (capacitance):
  kpex: 825
  magic: 897
```

In [3]:
```python
SUBSTRATE_NETS = {"VSUBS", "Vsubs", "vsubs", "GND", "SUB"}


def parse_cell_name(name):
    """Extract structure type, PDK, and parameters from GDS filename."""
    stem = name.replace(".gds", "")
    parts = stem.split("_")
    struct_type = parts[0]  # cross, mom, mim, overlap, sidewall, fringe,
    pdk = parts[1]          # ihp, sky130, gf180
    params = "_".join(parts[2:])
    return struct_type, pdk, params


def total_coupling_cap(cell_data, exclude_substrate=False):
    """Sum of unique off-diagonal capacitances (fF)."""
    matrix = cell_data.get("capacitance_matrix_fF", {})
    ports = cell_data.get("ports", [])
    if exclude_substrate:
        ports = [p for p in ports if p not in SUBSTRATE_NETS]
    total = 0.0
    for i, p1 in enumerate(ports):
        for j, p2 in enumerate(ports):
            if j > i:
                total += abs(matrix.get(p1, {}).get(p2, 0.0))
    return total


# Build main comparison DataFrame for cells present in both tools
all_cells = set(cap_data["kpex"]) & set(cap_data["magic"])
print(f"Cells common to both tools: {len(all_cells)}")
print(f"Exclude substrate capacitance: {EXCLUDE_SUBSTRATE}")

rows = []
for cell in sorted(all_cells):
    struct, pdk, params = parse_cell_name(cell)
    for tool in TOOLS:
        c = cap_data[tool].get(cell)
        if c is None:
            continue
        rows.append({
            "cell": cell,
            "structure": struct,
            "pdk": pdk,
            "params": params,
```

```
            "tool": tool,
            "total_coupling_fF": total_coupling_cap(c, exclude_substrate=
            "n_ports": len(c.get("ports", [])),
            "time_s": time_data[tool].get(cell, np.nan),
            "memory_MB": mem_data[tool].get(cell, np.nan),
        })

df = pd.DataFrame(rows)
df.head(10)
```

```
Cells common to both tools: 825
Exclude substrate capacitance: True
```

Out[3]:

	cell	structure	pdk	params	tool	total_
0	cross_gf180_L2_F10_W1x_S1x.gds	cross	gf180	L2_F10_W1x_S1x	kpex	
1	cross_gf180_L2_F10_W1x_S1x.gds	cross	gf180	L2_F10_W1x_S1x	magic	
2	cross_gf180_L2_F10_W1x_S2x.gds	cross	gf180	L2_F10_W1x_S2x	kpex	
3	cross_gf180_L2_F10_W1x_S2x.gds	cross	gf180	L2_F10_W1x_S2x	magic	
4	cross_gf180_L2_F10_W1x_S3x.gds	cross	gf180	L2_F10_W1x_S3x	kpex	
5	cross_gf180_L2_F10_W1x_S3x.gds	cross	gf180	L2_F10_W1x_S3x	magic	
6	cross_gf180_L2_F10_W2x_S1x.gds	cross	gf180	L2_F10_W2x_S1x	kpex	
7	cross_gf180_L2_F10_W2x_S1x.gds	cross	gf180	L2_F10_W2x_S1x	magic	
8	cross_gf180_L2_F10_W2x_S2x.gds	cross	gf180	L2_F10_W2x_S2x	kpex	
9	cross_gf180_L2_F10_W2x_S2x.gds	cross	gf180	L2_F10_W2x_S2x	magic	

1. Substrate Capacitance

Substrate structures consist of a **single conductor on one metal layer** with no adjacent conductors, so lateral (sidewall) and inter-layer (overlap) coupling are zero. The only capacitance present is from the wire to the silicon substrate.

Each tool encodes substrate capacitance differently:

- **kpex** — reports a `VSUBS` port; substrate cap is the off-diagonal entry between `NET1` and `VSUBS`.
- **MAGIC** — substrate cap is the self-term diagonal of each signal node (extracted from `node` lines in the `.ext` file).

Structures sweep: 5 metal layers × 4 width multipliers [1×, 2×, 5×, 10×] per PDK.

In [4]:

```
def extract_substrate_cap(cell_data, tool):
    """Return wire-to-substrate capacitance (fF) for a single-conductor c
    matrix = cell_data.get("capacitance_matrix_fF", {})
    ports = cell_data.get("ports", [])

    if tool == "kpex":
        sig = [p for p in ports if p not in SUBSTRATE_NETS]
        sub = [p for p in ports if p in SUBSTRATE_NETS]
        if not sig or not sub:
```

```
            return np.nan
        return sum(abs(matrix.get(s, {}).get(b, 0.0)) for s in sig for b
    else:  # magic: substrate cap stored as self-term diagonal
        sig = [p for p in ports if p not in SUBSTRATE_NETS]
        if not sig:
            return np.nan
        return sum(abs(matrix.get(p, {}).get(p, 0.0)) for p in sig)


substrate_cells = set()
for tool_data in cap_data.values():
    substrate_cells |= {c for c in tool_data if c.startswith("substrate_"

sub_rows = []
for cell in sorted(substrate_cells):
    struct, pdk, params = parse_cell_name(cell)
    for tool in TOOLS:
        c = cap_data[tool].get(cell)
        if c is None:
            continue
        sub_rows.append({
            "cell": cell,
            "pdk": pdk,
            "params": params,
            "tool": tool,
            "substrate_cap_fF": extract_substrate_cap(c, tool),
            "time_s": time_data[tool].get(cell, np.nan),
            "memory_MB": mem_data[tool].get(cell, np.nan),
        })

df_sub = pd.DataFrame(sub_rows)
df_sub["metal"] = df_sub["params"].str.extract(r"M(\d+)").astype(int)
df_sub["width_mult"] = df_sub["params"].str.extract(r"W(\d+)x").astype(in

print(f"Substrate structures: {len(substrate_cells)} cells, {len(df_sub)}
df_sub.head()
```

```
Substrate structures: 60 cells, 120 tool-cell rows
```

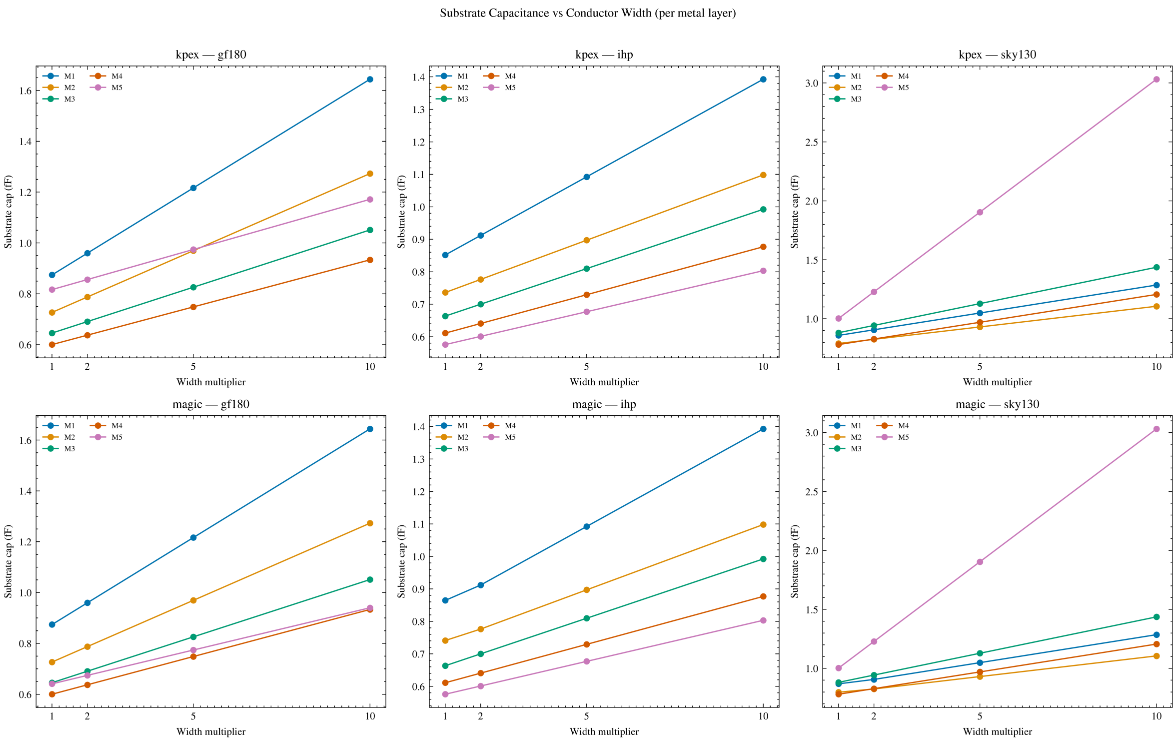
Out[4]:

	cell	pdk	params	tool	substrate_cap_fF	time_s
0	substrate_gf180_M1_W10x.gds	gf180	M1_W10x	kpex	1.643995	0.0181
1	substrate_gf180_M1_W10x.gds	gf180	M1_W10x	magic	1.643990	0.1099
2	substrate_gf180_M1_W1x.gds	gf180	M1_W1x	kpex	0.874157	0.0186
3	substrate_gf180_M1_W1x.gds	gf180	M1_W1x	magic	0.874157	0.1082
4	substrate_gf180_M1_W2x.gds	gf180	M1_W2x	kpex	0.959695	0.0166

In [5]:

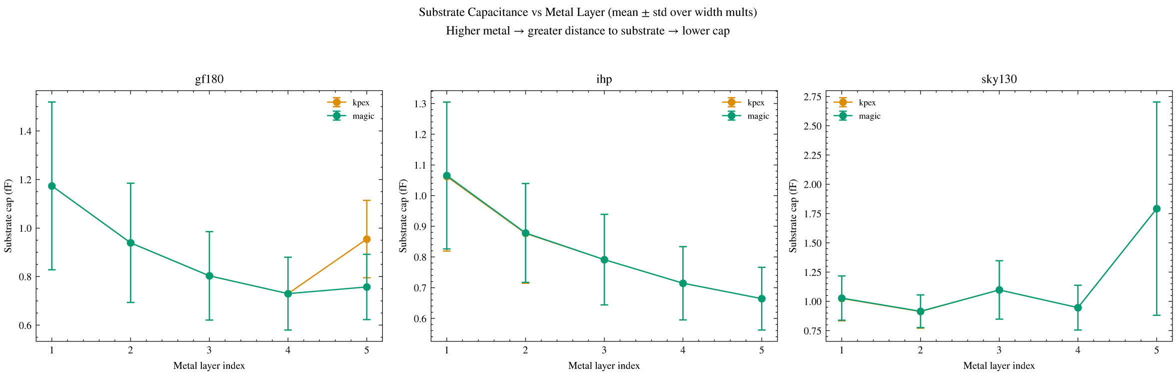
```
# Substrate cap vs width multiplier — one line per metal layer, subplot p
fig, axes = plt.subplots(2, 3, figsize=(13, 8), sharey=False)
tools_plot = ["kpex", "magic"]
pdks_sorted = sorted(df_sub["pdk"].unique())
width_mults = sorted(df_sub["width_mult"].unique())
colors = sns.color_palette("colorblind", n_colors=5)

for row_idx, tool in enumerate(tools_plot):
    for col_idx, pdk in enumerate(pdks_sorted):
        ax = axes[row_idx][col_idx]
```

```
        subset = df_sub[(df_sub["tool"] == tool) & (df_sub["pdk"] == pdk)
        for mi, metal_idx in enumerate(sorted(subset["metal"].unique())):
            ms = subset[subset["metal"] == metal_idx].sort_values("width_
            if ms.empty:
                continue
            ax.plot(
                ms["width_mult"], ms["substrate_cap_fF"],
                "o-", color=colors[mi], label=f"M{metal_idx}", markersize
            )
        ax.set_xlabel("Width multiplier")
        ax.set_ylabel("Substrate cap (fF)")
        ax.set_title(f"{tool} — {pdk}")
        ax.set_xticks(width_mults)
        ax.legend(fontsize=6, ncol=2)

fig.suptitle("Substrate Capacitance vs Conductor Width (per metal layer)"
fig.tight_layout()
plt.show()
```



In [6]:
```
# Substrate cap vs metal layer — shows height dependence; averaged over w
fig, axes = plt.subplots(1, 3, figsize=(13, 4))
tool_colors = {"kpex": "C1", "magic": "C2"}

for ax, pdk in zip(axes, pdks_sorted):
    subset = df_sub[df_sub["pdk"] == pdk]
    for tool in tools_plot:
        ts = subset[subset["tool"] == tool].dropna(subset=["substrate_cap
        if ts.empty:
            continue
        grouped = ts.groupby("metal")["substrate_cap_fF"].agg(["mean", "s
        ax.errorbar(
            grouped.index, grouped["mean"],
            yerr=grouped["std"],
            fmt="o-", capsize=3, label=tool,
            color=tool_colors[tool], markersize=5,
        )
    ax.set_xlabel("Metal layer index")
```

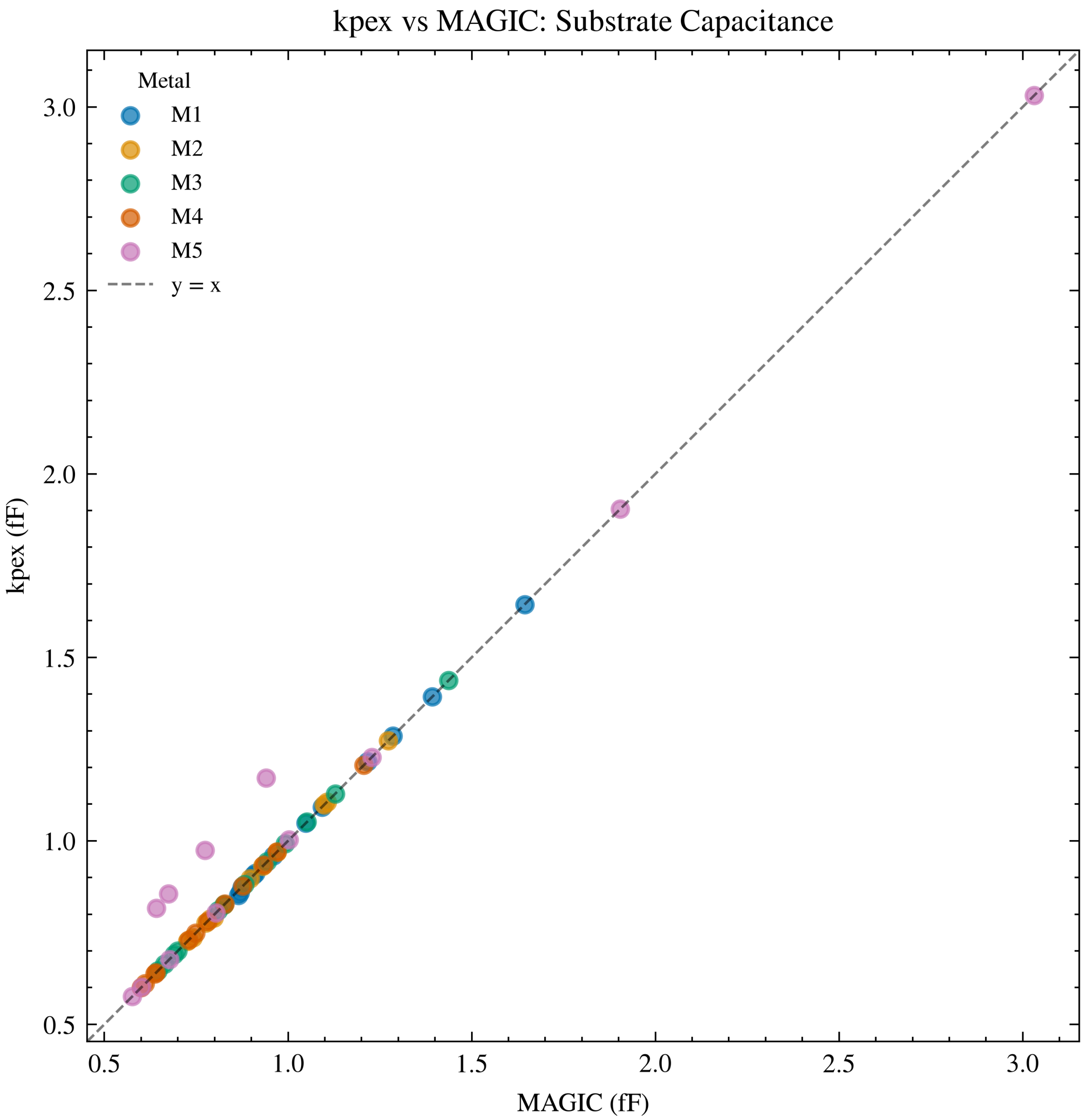
```
    ax.set_ylabel("Substrate cap (fF)")
    ax.set_title(pdk)
    ax.set_xticks(range(1, 6))
    ax.legend(fontsize=7)

fig.suptitle(
    "Substrate Capacitance vs Metal Layer (mean ± std over width mults)\n
    "Higher metal → greater distance to substrate → lower cap",
    y=1.03,
)
fig.tight_layout()
plt.show()
```



In [7]:

```
# kpex vs magic scatter: substrate capacitance coloured by metal layer
sub_pivot = df_sub.pivot_table(
    index=["cell", "pdk", "metal", "width_mult"],
    columns="tool",
    values="substrate_cap_fF",
).reset_index()

fig, ax = plt.subplots(figsize=(6, 5))
colors_metal = sns.color_palette("colorblind", n_colors=5)
for mi in range(1, 6):
    sub = sub_pivot[sub_pivot["metal"] == mi].dropna(subset=["kpex", "mag
    if sub.empty:
        continue
    ax.scatter(sub["magic"], sub["kpex"], color=colors_metal[mi - 1],
               alpha=0.7, s=25, label=f"M{mi}")

lims_min = min(ax.get_xlim()[0], ax.get_ylim()[0])
lims_max = max(ax.get_xlim()[1], ax.get_ylim()[1])
ax.plot([lims_min, lims_max], [lims_min, lims_max], "k--", lw=0.8, alpha=
ax.set_xlim(lims_min, lims_max)
ax.set_ylim(lims_min, lims_max)
ax.set_xlabel("MAGIC (fF)")
ax.set_ylabel("kpex (fF)")
ax.set_title("kpex vs MAGIC: Substrate Capacitance")
ax.set_aspect("equal")
ax.legend(fontsize=7, title="Metal", title_fontsize=7)
fig.tight_layout()
plt.show()

# Relative error statistics
valid = sub_pivot[["kpex", "magic"]].dropna()
valid = valid[valid["magic"] > 0]
rel_err = (valid["kpex"] - valid["magic"]) / valid["magic"] * 100
print(f"kpex vs MAGIC (substrate):")
```

```
print(f"  Mean: {rel_err.mean():.1f}%  Median: {rel_err.median():.1f}%"
      f"  Std: {rel_err.std():.1f}%  MAE: {rel_err.abs().mean():.1f}%")
```




```
kpex vs MAGIC (substrate):
  Mean: 1.7%  Median: -0.0%  Std: 6.6%  MAE: 1.8%
```

In [8]:
```
# Summary table: substrate cap by PDK and metal layer (mean ± std over wi
sub_summary = (
    df_sub.dropna(subset=["substrate_cap_fF"])
    .groupby(["pdk", "metal", "tool"])["substrate_cap_fF"]
    .agg(mean="mean", std="std")
    .round(4)
    .reset_index()
)

sub_wide = sub_summary.pivot_table(
    index=["pdk", "metal"],
    columns="tool",
    values=["mean", "std"],
).round(4)
sub_wide.columns = [f"{col[1]} {col[0]}" for col in sub_wide.columns]
sub_wide = sub_wide[[c for c in sorted(sub_wide.columns)]]
print("Substrate capacitance (fF) by PDK and metal layer (mean ± std over
print(sub_wide.to_string())
```

```
Substrate capacitance (fF) by PDK and metal layer (mean ± std over width m
ultipliers):

              kpex mean  kpex std  magic mean  magic std
pdk    metal
gf180  1         1.1735    0.3457      1.1735     0.3457
       2         0.9391    0.2453      0.9391     0.2453
       3         0.8033    0.1822      0.8033     0.1822
       4         0.7298    0.1497      0.7298     0.1497
       5         0.9545    0.1594      0.7573     0.1344
ihp    1         1.0622    0.2429      1.0654     0.2393
       2         0.8770    0.1626      0.8782     0.1611
       3         0.7914    0.1476      0.7914     0.1476
       4         0.7145    0.1193      0.7145     0.1193
       5         0.6643    0.1022      0.6643     0.1022
sky130 1         1.0249    0.1918      1.0275     0.1889
       2         0.9130    0.1417      0.9149     0.1396
       3         1.0975    0.2494      1.0975     0.2494
       4         0.9463    0.1910      0.9463     0.1910
       5         1.7915    0.9111      1.7915     0.9111
```

2. Overlap Capacitance

Overlap structures have a **single metal line per layer**, all stacked at the same position. No adjacent conductors on the same layer means zero sidewall capacitance — only overlap (parallel-plate) capacitance between vertically stacked layers is present.

This section validates extractor overlap models by comparing total coupling capacitance as a function of layer count and conductor width.

In [9]:
```python
overlap_cells = set()
for tool_data in cap_data.values():
    overlap_cells |= {c for c in tool_data if c.startswith("overlap_")}

overlap_rows = []
for cell in sorted(overlap_cells):
    struct, pdk, params = parse_cell_name(cell)
    for tool in TOOLS:
        c = cap_data[tool].get(cell)
        if c is None:
            continue
        overlap_rows.append({
            "cell": cell,
            "pdk": pdk,
            "params": params,
            "tool": tool,
            "total_coupling_fF": total_coupling_cap(c, exclude_substrate=
            "n_ports": len([p for p in c.get("ports", []) if p not in SUB
        })

df_overlap = pd.DataFrame(overlap_rows)
df_overlap["n_layers"] = df_overlap["params"].str.extract(r"L(\d+)").asty
df_overlap["width_mult"] = df_overlap["params"].str.extract(r"W(\d+)x").a

print(f"Overlap structures: {len(overlap_cells)} cells, {len(df_overlap)}
df_overlap.head()
```

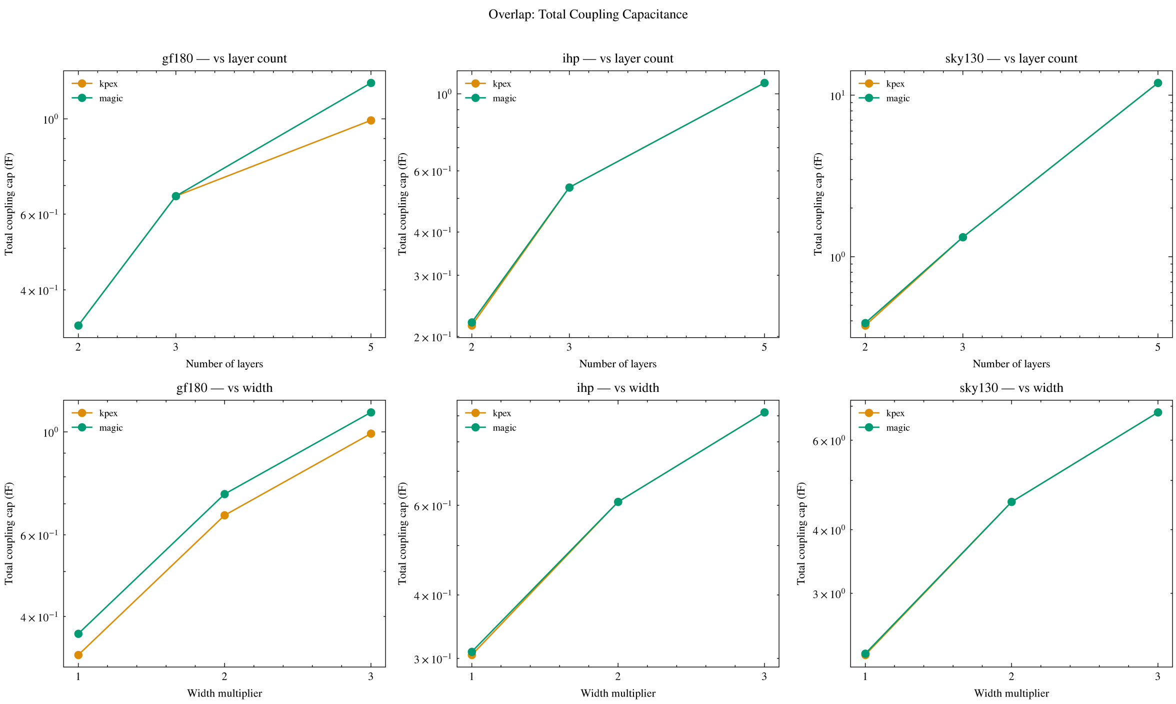
Overlap structures: 27 cells, 54 tool-cell rows

Out[9]:

	cell	pdk	params	tool	total_coupling_fF	n_ports	n_l
0	overlap_gf180_L2_W1x.gds	gf180	L2_W1x	kpex	0.165276	2	
1	overlap_gf180_L2_W1x.gds	gf180	L2_W1x	magic	0.165276	2	
2	overlap_gf180_L2_W2x.gds	gf180	L2_W2x	kpex	0.330551	2	
3	overlap_gf180_L2_W2x.gds	gf180	L2_W2x	magic	0.330551	2	
4	overlap_gf180_L2_W3x.gds	gf180	L2_W3x	kpex	0.495827	2	

In [10]:

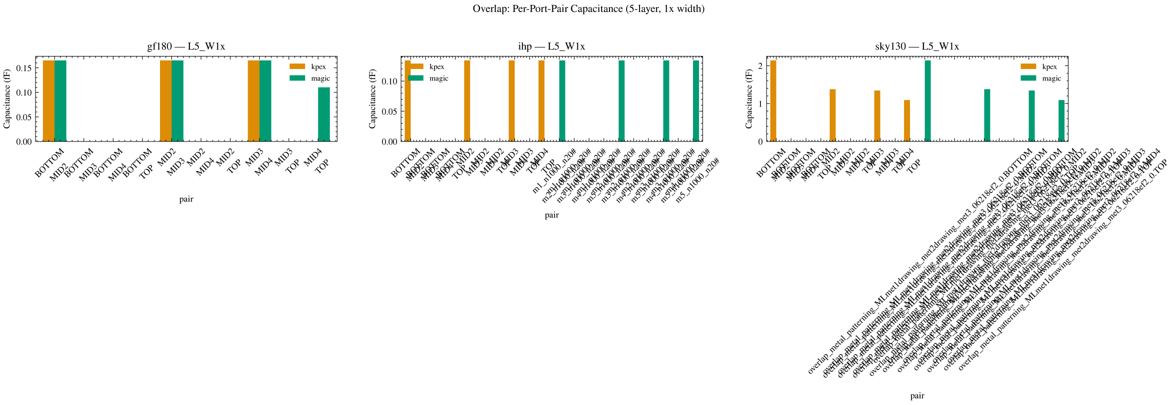
```
# Overlap: total coupling cap vs layer count and width multiplier
fig, axes = plt.subplots(2, 3, figsize=(12, 7), sharey=False)
pdks_sorted = sorted(df_overlap["pdk"].unique())
tool_colors = {"kpex": "C1", "magic": "C2"}

for col_idx, pdk in enumerate(pdks_sorted):
    subset = df_overlap[df_overlap["pdk"] == pdk]

    ax = axes[0][col_idx]
    for tool in TOOLS:
        ts = subset[subset["tool"] == tool]
        if ts.empty:
            continue
        grouped = ts.groupby("n_layers")["total_coupling_fF"].mean()
        ax.plot(grouped.index, grouped.values, "o-", label=tool,
                color=tool_colors[tool], markersize=5)
    ax.set_xlabel("Number of layers")
    ax.set_ylabel("Total coupling cap (fF)")
    ax.set_title(f"{pdk} — vs layer count")
    ax.legend(fontsize=7)
    ax.set_xticks([2, 3, 5])
    ax.set_yscale("log")

    ax = axes[1][col_idx]
    for tool in TOOLS:
        ts = subset[subset["tool"] == tool]
        if ts.empty:
            continue
        grouped = ts.groupby("width_mult")["total_coupling_fF"].mean()
        ax.plot(grouped.index, grouped.values, "o-", label=tool,
                color=tool_colors[tool], markersize=5)
    ax.set_xlabel("Width multiplier")
    ax.set_ylabel("Total coupling cap (fF)")
    ax.set_title(f"{pdk} — vs width")
    ax.legend(fontsize=7)
    ax.set_xticks([1, 2, 3])
    ax.set_yscale("log")

fig.suptitle("Overlap: Total Coupling Capacitance", y=1.01)
fig.tight_layout()
plt.show()
```



In [11]:

```
# Overlap: per-port-pair capacitance breakdown for 5-layer structures at
target_params = "L5_W1x"
breakdown_rows = []

for cell in sorted(overlap_cells):
    struct, pdk, params = parse_cell_name(cell)
    if params != target_params:
        continue
    for tool in TOOLS:
        c = cap_data[tool].get(cell)
        if c is None:
            continue
        ports = sorted(p for p in c.get("ports", []) if p not in SUBSTRAT
        matrix = c.get("capacitance_matrix_fF", {})
        for i, p1 in enumerate(ports):
            for j, p2 in enumerate(ports):
                if j > i:
                    cap = abs(matrix.get(p1, {}).get(p2, 0.0))
                    breakdown_rows.append({
                        "pdk": pdk, "tool": tool,
                        "pair": f"{p1}\n{p2}", "cap_fF": cap,
                    })

df_breakdown = pd.DataFrame(breakdown_rows)
if not df_breakdown.empty:
    pdks = sorted(df_breakdown["pdk"].unique())
    fig, axes = plt.subplots(1, len(pdks), figsize=(5 * len(pdks), 5))
    if len(pdks) == 1:
        axes = [axes]
    for ax, pdk in zip(axes, pdks):
        subset = df_breakdown[df_breakdown["pdk"] == pdk]
        if subset.empty:
            continue
        pivot = subset.pivot_table(index="pair", columns="tool", values="
        pivot.plot(kind="bar", ax=ax, width=0.8, color=[tool_colors[t] fo
        ax.set_ylabel("Capacitance (fF)")
        ax.set_title(f"{pdk} — {target_params}")
        ax.legend(fontsize=7)
```

```python
        ax.tick_params(axis="x", rotation=45)
    fig.suptitle("Overlap: Per-Port-Pair Capacitance (5-layer, 1x width)"
    fig.tight_layout()
    plt.show()
else:
    print("No overlap L5_W1x data found.")
```



3. Sidewall Capacitance

Sidewall structures have **multiple conductors on a single metal layer**, each assigned a separate net. No second metal layer is present, so vertical (overlap) capacitance is zero — only lateral (sidewall) coupling between adjacent conductors is measured.

This section validates extractor sidewall models by comparing total coupling capacitance as a function of spacing, width, number of conductors, and metal layer.

In [12]:

```python
sidewall_cells = set()
for tool_data in cap_data.values():
    sidewall_cells |= {c for c in tool_data if c.startswith("sidewall_")}

sidewall_rows = []
for cell in sorted(sidewall_cells):
    struct, pdk, params = parse_cell_name(cell)
    for tool in TOOLS:
        c = cap_data[tool].get(cell)
        if c is None:
            continue
        sidewall_rows.append({
            "cell": cell,
            "pdk": pdk,
            "params": params,
            "tool": tool,
            "total_coupling_fF": total_coupling_cap(c, exclude_substrate=
            "n_ports": len([p for p in c.get("ports", []) if p not in SUB
        })

df_sidewall = pd.DataFrame(sidewall_rows)
df_sidewall["metal"] = df_sidewall["params"].str.extract(r"M(\d+)").astyp
df_sidewall["n_conductors"] = df_sidewall["params"].str.extract(r"N(\d+)"
df_sidewall["width_mult"] = df_sidewall["params"].str.extract(r"W(\d+)x")
df_sidewall["spacing_mult"] = df_sidewall["params"].str.extract(r"S(\d+)x
```

```python
print(f"Sidewall structures: {len(sidewall_cells)} cells, {len(df_sidewal
df_sidewall.head()
```

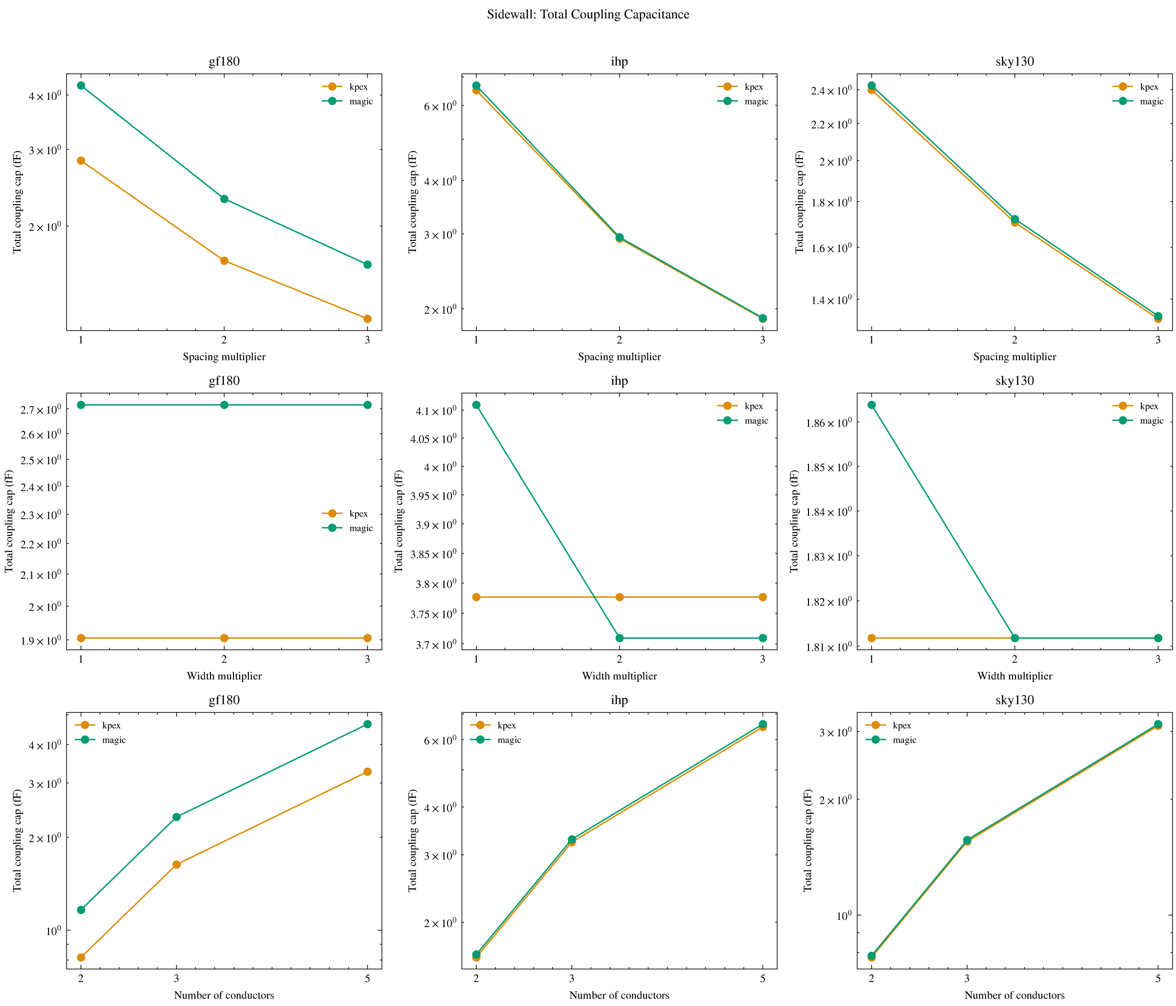
```
Sidewall structures: 405 cells, 783 tool-cell rows
```

Out[12]:

	cell	pdk	params	tool	total_coupling
0	sidewall_gf180_M1_N2_W1x_S1x.gds	gf180	M1_N2_W1x_S1x	kpex	2.288
1	sidewall_gf180_M1_N2_W1x_S1x.gds	gf180	M1_N2_W1x_S1x	magic	2.250
2	sidewall_gf180_M1_N2_W1x_S2x.gds	gf180	M1_N2_W1x_S2x	kpex	0.995
3	sidewall_gf180_M1_N2_W1x_S2x.gds	gf180	M1_N2_W1x_S2x	magic	0.988
4	sidewall_gf180_M1_N2_W1x_S3x.gds	gf180	M1_N2_W1x_S3x	kpex	0.635

In [13]:

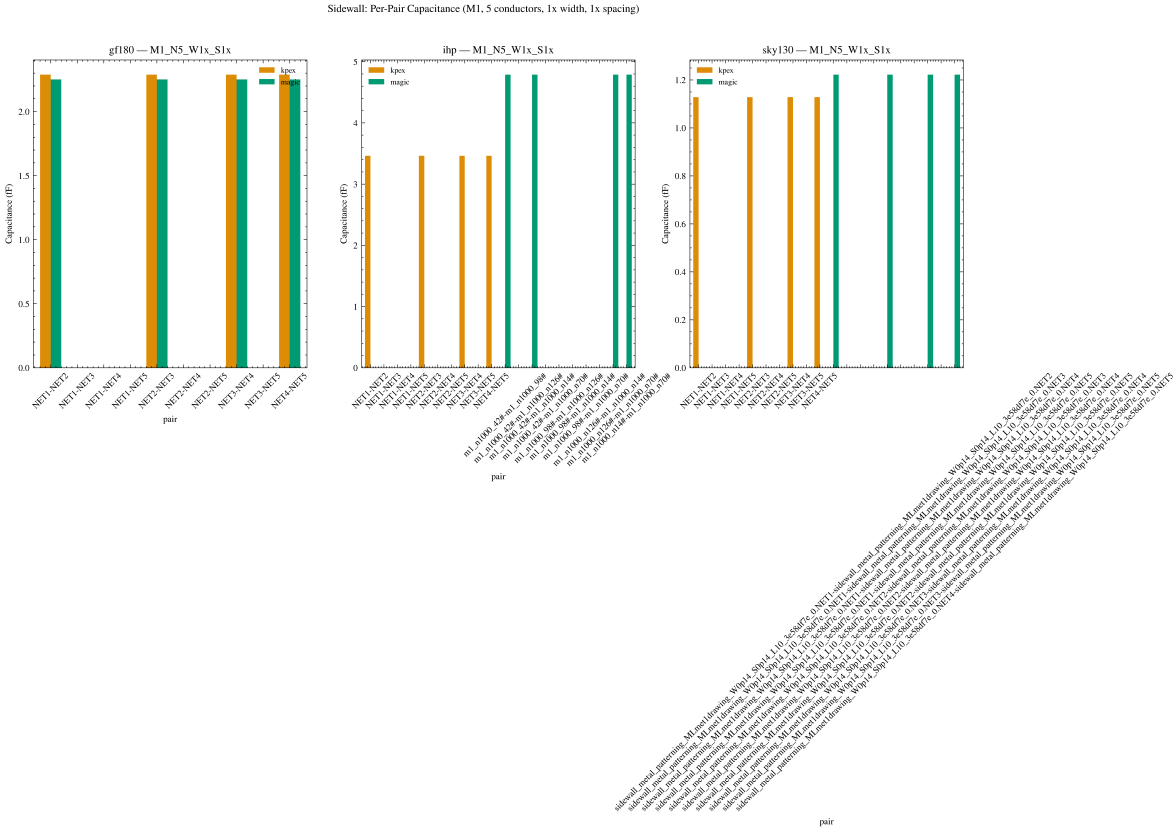
```python
# Sidewall: total coupling cap vs spacing, width, and number of conductor
fig, axes = plt.subplots(3, 3, figsize=(12, 10), sharey=False)
pdks_sorted = sorted(df_sidewall["pdk"].unique())
dims = [
    ("spacing_mult", "Spacing multiplier", [1, 2, 3]),
    ("width_mult",   "Width multiplier",   [1, 2, 3]),
    ("n_conductors", "Number of conductors", [2, 3, 5]),
]

for row_idx, (col, xlabel, xticks) in enumerate(dims):
    for col_idx, pdk in enumerate(pdks_sorted):
        ax = axes[row_idx][col_idx]
        subset = df_sidewall[df_sidewall["pdk"] == pdk]
        for tool in TOOLS:
            ts = subset[subset["tool"] == tool]
            if ts.empty:
                continue
            grouped = ts.groupby(col)["total_coupling_fF"].mean()
            ax.plot(grouped.index, grouped.values, "o-", label=tool,
                    color=tool_colors[tool], markersize=5)
        ax.set_xlabel(xlabel)
        ax.set_ylabel("Total coupling cap (fF)")
        ax.set_title(pdk)
        ax.legend(fontsize=7)
        ax.set_xticks(xticks)
        ax.set_yscale("log")

fig.suptitle("Sidewall: Total Coupling Capacitance", y=1.01)
fig.tight_layout()
plt.show()
```




```
In [14]: # Sidewall: per-pair capacitance breakdown — M1, 5 conductors, 1x width,
sw_target = "M1_N5_W1x_S1x"
sw_breakdown_rows = []

for cell in sorted(sidewall_cells):
    struct, pdk, params = parse_cell_name(cell)
    if params != sw_target:
        continue
    for tool in TOOLS:
        c = cap_data[tool].get(cell)
        if c is None:
            continue
        ports = sorted(p for p in c.get("ports", []) if p not in SUBSTRAT
        matrix = c.get("capacitance_matrix_fF", {})
        for i, p1 in enumerate(ports):
            for j, p2 in enumerate(ports):
                if j > i:
                    cap = abs(matrix.get(p1, {}).get(p2, 0.0))
                    sw_breakdown_rows.append({
                        "pdk": pdk, "tool": tool,
                        "pair": f"{p1}-{p2}", "cap_fF": cap,
                    })

df_sw_breakdown = pd.DataFrame(sw_breakdown_rows)
if not df_sw_breakdown.empty:
    pdks = sorted(df_sw_breakdown["pdk"].unique())
    fig, axes = plt.subplots(1, len(pdks), figsize=(5 * len(pdks), 5))
    if len(pdks) == 1:
        axes = [axes]
```

```
    for ax, pdk in zip(axes, pdks):
        subset = df_sw_breakdown[df_sw_breakdown["pdk"] == pdk]
        if subset.empty:
            continue
        pivot = subset.pivot_table(index="pair", columns="tool", values="
        pivot.plot(kind="bar", ax=ax, width=0.8, color=[tool_colors[t] fo
        ax.set_ylabel("Capacitance (fF)")
        ax.set_title(f"{pdk} — {sw_target}")
        ax.legend(fontsize=7)
        ax.tick_params(axis="x", rotation=45)
    fig.suptitle("Sidewall: Per-Pair Capacitance (M1, 5 conductors, 1x wi
    fig.tight_layout()
    plt.show()
else:
    print(f"No sidewall {sw_target} data found.")
```

```
/var/folders/58/9x4h23m57hl3nj4g9kjnldhw0000gn/T/ipykernel_79930/278322817
0.py:41: UserWarning: Tight layout not applied. The bottom and top margins
cannot be made large enough to accommodate all Axes decorations.
  fig.tight_layout()
```



4. Fringe / Side-Overlap Capacitance

Fringe structures use **three consecutive metal layers** with staggered conductors: conductors on the middle layer alternate with conductors on the bottom and top layers. No direct vertical overlap exists between mid and outer conductors, so the dominant coupling is fringe-field (side-overlap) capacitance from conductor edges.

This section validates extractor fringe models by comparing total coupling capacitance as a function of spacing, width, and number of conductors across metal layer ranges.

In [15]:

```
fringe_cells = set()
for tool_data in cap_data.values():
    fringe_cells |= {c for c in tool_data if c.startswith("fringe_")}

fringe_rows = []
for cell in sorted(fringe_cells):
    struct, pdk, params = parse_cell_name(cell)
    for tool in TOOLS:
        c = cap_data[tool].get(cell)
        if c is None:
            continue
        fringe_rows.append({
            "cell": cell,
            "pdk": pdk,
            "params": params,
            "tool": tool,
            "total_coupling_fF": total_coupling_cap(c, exclude_substrate=
            "n_ports": len([p for p in c.get("ports", []) if p not in SUB
        })

df_fringe = pd.DataFrame(fringe_rows)
df_fringe["metal_range"] = df_fringe["params"].str.extract(r"(M\d+-\d+)")
df_fringe["n_conductors"] = df_fringe["params"].str.extract(r"N(\d+)").as
df_fringe["width_mult"] = df_fringe["params"].str.extract(r"W(\d+)x").ast
df_fringe["spacing_mult"] = df_fringe["params"].str.extract(r"S(\d+)x").a

print(f"Fringe structures: {len(fringe_cells)} cells, {len(df_fringe)} to
df_fringe.head()
```

Fringe structures: 243 cells, 459 tool-cell rows

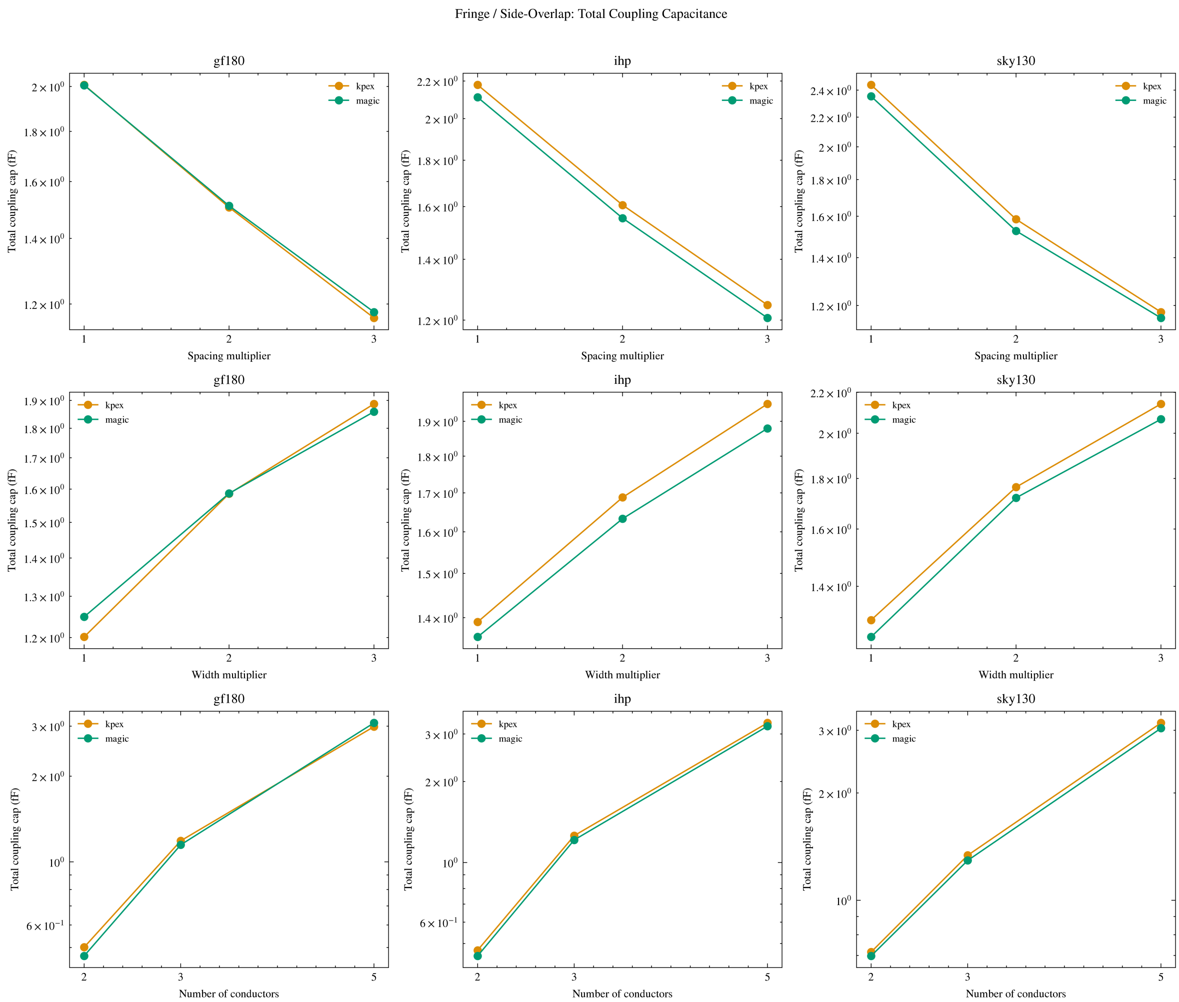
Out[15]:

	cell	pdk	params	tool	total_coupling_fF	n_ports	me
0	fringe_gf180_M1-3_N2_W1x_S1x.gds	gf180	M1-3_N2_W1x_S1x	kpex	0.355405	3	
1	fringe_gf180_M1-3_N2_W1x_S1x.gds	gf180	M1-3_N2_W1x_S1x	magic	0.335333	3	
2	fringe_gf180_M1-3_N2_W1x_S2x.gds	gf180	M1-3_N2_W1x_S2x	kpex	0.278910	3	
3	fringe_gf180_M1-3_N2_W1x_S2x.gds	gf180	M1-3_N2_W1x_S2x	magic	0.263978	3	
4	fringe_gf180_M1-3_N2_W1x_S3x.gds	gf180	M1-3_N2_W1x_S3x	kpex	0.216876	3	

In [16]:

```
# Fringe: total coupling cap vs spacing, width, and number of conductors
fig, axes = plt.subplots(3, 3, figsize=(12, 10), sharey=False)
pdks_sorted = sorted(df_fringe["pdk"].unique())
dims = [
    ("spacing_mult", "Spacing multiplier", [1, 2, 3]),
    ("width_mult",   "Width multiplier",   [1, 2, 3]),
    ("n_conductors", "Number of conductors", [2, 3, 5]),
]

for row_idx, (col, xlabel, xticks) in enumerate(dims):
    for col_idx, pdk in enumerate(pdks_sorted):
        ax = axes[row_idx][col_idx]
        subset = df_fringe[df_fringe["pdk"] == pdk]
```

```
        for tool in TOOLS:
            ts = subset[subset["tool"] == tool]
            if ts.empty:
                continue
            grouped = ts.groupby(col)["total_coupling_fF"].mean()
            ax.plot(grouped.index, grouped.values, "o-", label=tool,
                    color=tool_colors[tool], markersize=5)
        ax.set_xlabel(xlabel)
        ax.set_ylabel("Total coupling cap (fF)")
        ax.set_title(pdk)
        ax.legend(fontsize=7)
        ax.set_xticks(xticks)
        ax.set_yscale("log")

fig.suptitle("Fringe / Side-Overlap: Total Coupling Capacitance", y=1.01)
fig.tight_layout()
plt.show()
```

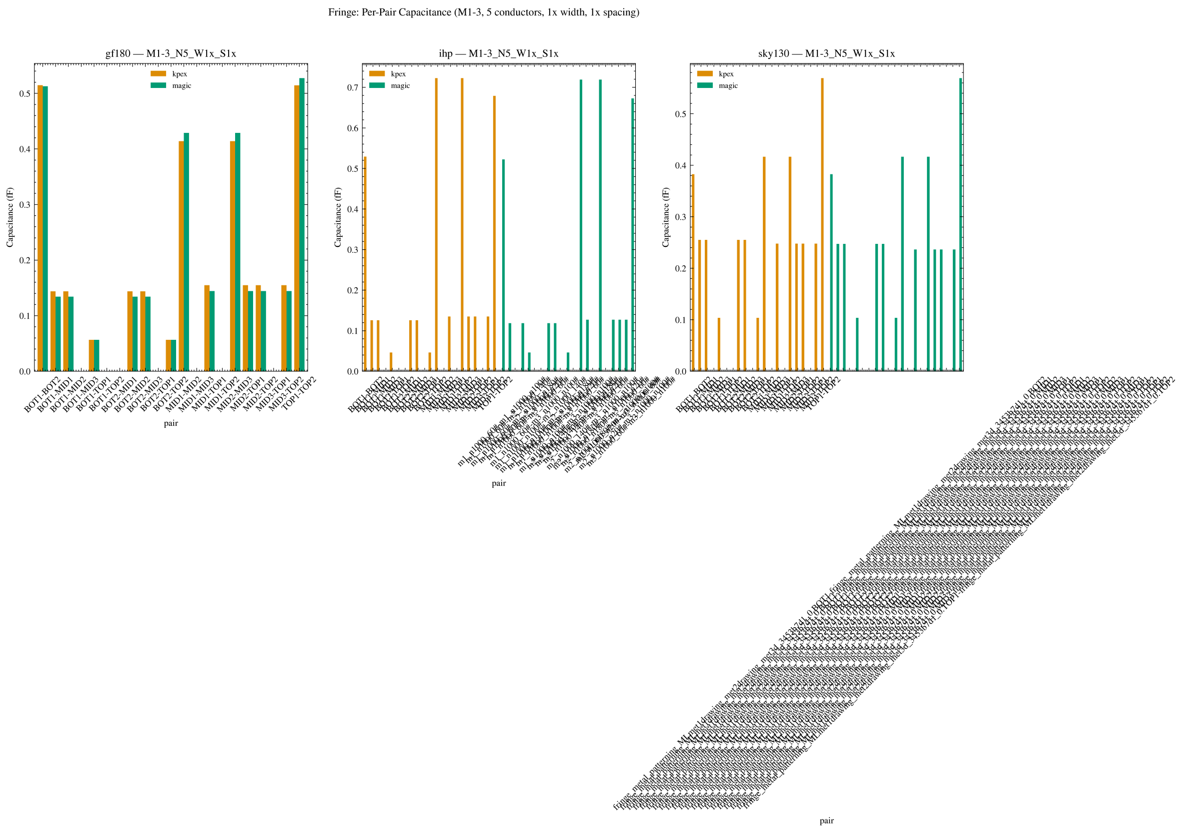


In [17]:

```
# Fringe: per-pair breakdown for M1-3, 5 conductors, 1x width, 1x spacing
fr_target = "M1-3_N5_W1x_S1x"
fr_breakdown_rows = []

for cell in sorted(fringe_cells):
    struct, pdk, params = parse_cell_name(cell)
    if params != fr_target:
        continue
    for tool in TOOLS:
        c = cap_data[tool].get(cell)
        if c is None:
            continue
```

```
        ports = sorted(p for p in c.get("ports", []) if p not in SUBSTRAT
        matrix = c.get("capacitance_matrix_fF", {})
        for i, p1 in enumerate(ports):
            for j, p2 in enumerate(ports):
                if j > i:
                    cap = abs(matrix.get(p1, {}).get(p2, 0.0))
                    fr_breakdown_rows.append({
                        "pdk": pdk, "tool": tool,
                        "pair": f"{p1}-{p2}", "cap_fF": cap,
                    })

df_fr_breakdown = pd.DataFrame(fr_breakdown_rows)
if not df_fr_breakdown.empty:
    pdks = sorted(df_fr_breakdown["pdk"].unique())
    fig, axes = plt.subplots(1, len(pdks), figsize=(5 * len(pdks), 5))
    if len(pdks) == 1:
        axes = [axes]
    for ax, pdk in zip(axes, pdks):
        subset = df_fr_breakdown[df_fr_breakdown["pdk"] == pdk]
        if subset.empty:
            continue
        pivot = subset.pivot_table(index="pair", columns="tool", values="
        pivot.plot(kind="bar", ax=ax, width=0.8, color=[tool_colors[t] fo
        ax.set_ylabel("Capacitance (fF)")
        ax.set_title(f"{pdk} — {fr_target}")
        ax.legend(fontsize=7)
        ax.tick_params(axis="x", rotation=45)
    fig.suptitle("Fringe: Per-Pair Capacitance (M1-3, 5 conductors, 1x wi
    fig.tight_layout()
    plt.show()
else:
    print(f"No fringe {fr_target} data found.")
```

```
/var/folders/58/9x4h23m57hl3nj4g9kjnldhw0000gn/T/ipykernel_79930/110515053
4.py:41: UserWarning: Tight layout not applied. The bottom and top margins
cannot be made large enough to accommodate all Axes decorations.
  fig.tight_layout()
```



5. Total Coupling & Cross Structures

Cross structures use **perpendicular metal lines on alternating layers** (BOTTOM/MID/TOP ports) to capture inter-layer coupling that combines overlap, sidewall, and fringe mechanisms. This section compares kpex vs MAGIC across all structure types using total coupling capacitance.

In [18]:

```python
# Pivot to get per-tool columns
cap_pivot = df.pivot_table(
    index=["cell", "structure", "pdk", "params"],
    columns="tool",
    values="total_coupling_fF",
).reset_index()

cap_pivot.head()
```

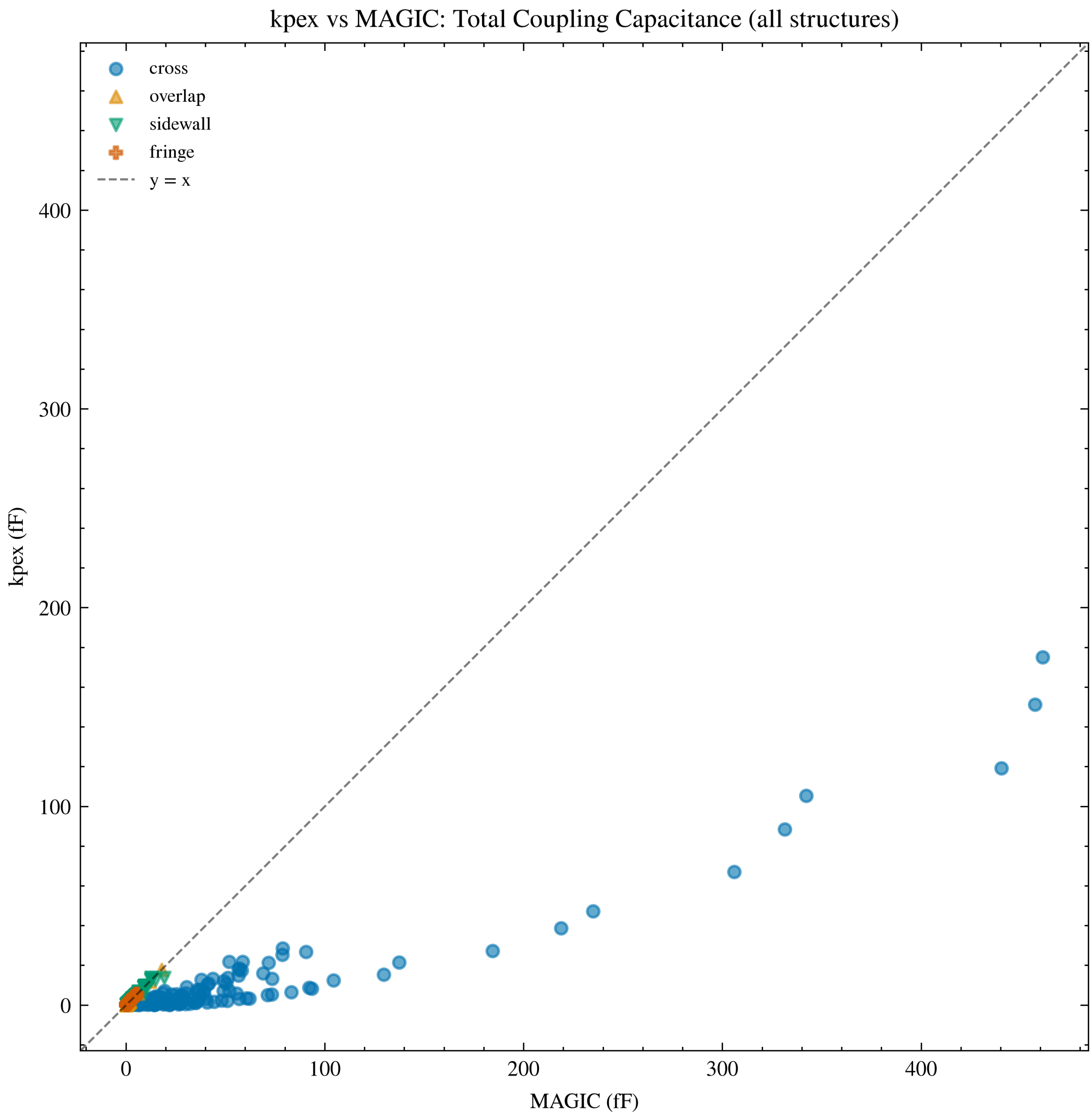
Out[18]:

tool	cell	structure	pdk	params	kpex
0	cross_gf180_L2_F10_W1x_S1x.gds	cross	gf180	L2_F10_W1x_S1x	1.164020
1	cross_gf180_L2_F10_W1x_S2x.gds	cross	gf180	L2_F10_W1x_S2x	1.798138
2	cross_gf180_L2_F10_W1x_S3x.gds	cross	gf180	L2_F10_W1x_S3x	2.285079
3	cross_gf180_L2_F10_W2x_S1x.gds	cross	gf180	L2_F10_W2x_S1x	3.136526
4	cross_gf180_L2_F10_W2x_S2x.gds	cross	gf180	L2_F10_W2x_S2x	4.393775

In [33]:

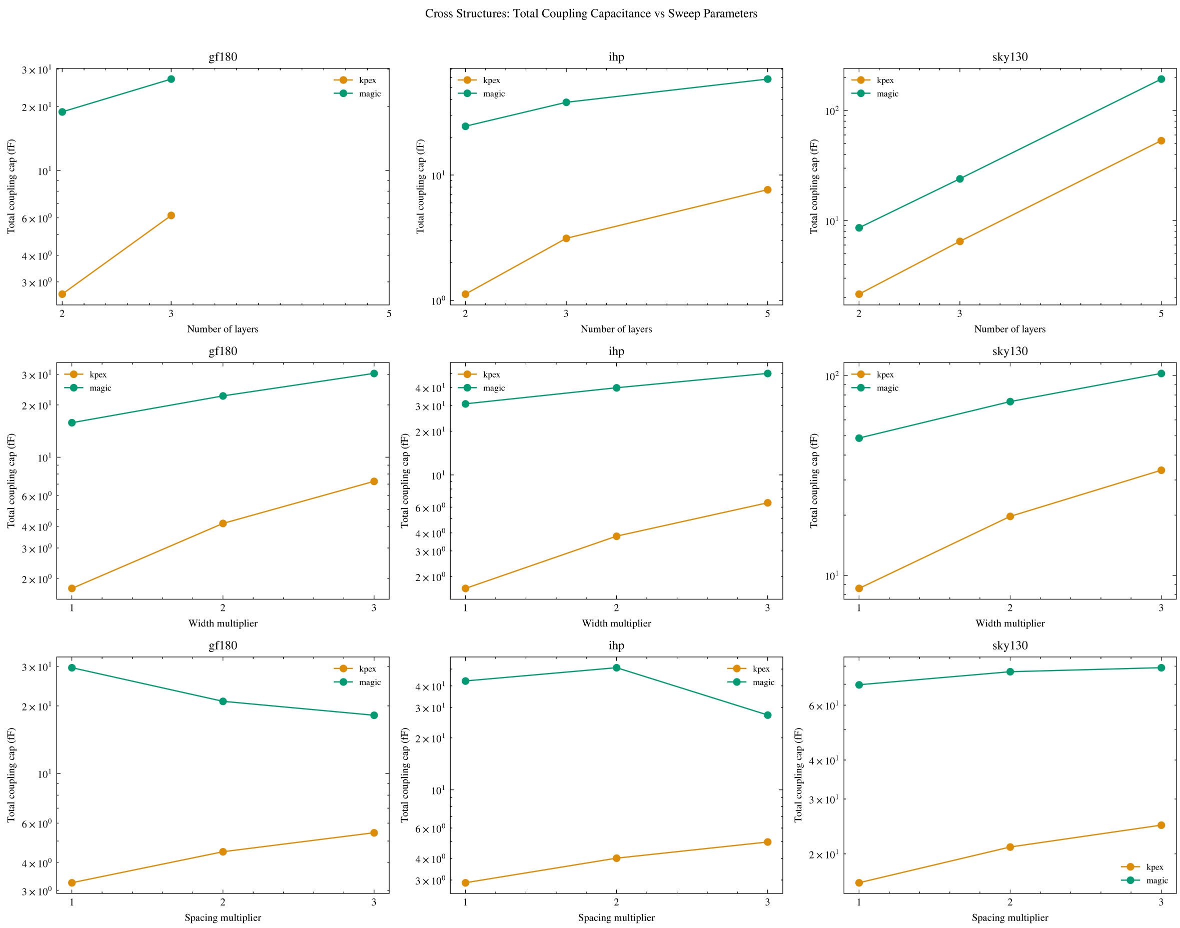
```python
# kpex vs magic scatter — all structure types
fig, ax = plt.subplots(figsize=(7, 6))
markers = {"cross": "o", "mom": "s", "mim": "D", "overlap": "^", "sidewal
```

```
for struct, marker in markers.items():
    mask = cap_pivot["structure"] == struct
    subset = cap_pivot[mask]
    if subset.empty or "kpex" not in subset.columns or "magic" not in sub
        continue
    ax.scatter(subset["magic"], subset["kpex"], marker=marker,
               alpha=0.6, s=20, label=struct)

lims = [min(ax.get_xlim()[0], ax.get_ylim()[0]), max(ax.get_xlim()[1], ax
ax.plot(lims, lims, "k--", lw=0.8, alpha=0.5, label="y = x")
ax.set_xlim(lims)
ax.set_ylim(lims)
ax.set_xlabel("MAGIC (fF)")
ax.set_ylabel("kpex (fF)")
ax.set_title("kpex vs MAGIC: Total Coupling Capacitance (all structures)"
ax.set_aspect("equal")
ax.legend(fontsize=7)
fig.tight_layout()
plt.show()
```



In [34]:
```
# kpex vs magic: cross structures only — sweep params
df_cross = df[df["structure"] == "cross"].copy()
if not df_cross.empty:
    df_cross["n_layers"] = df_cross["params"].str.extract(r"L(\d+)").asty
```

```python
df_cross["n_fingers"] = df_cross["params"].str.extract(r"F(\d+)").ast
df_cross["width_mult"] = df_cross["params"].str.extract(r"W(\d+)x").a
df_cross["spacing_mult"] = df_cross["params"].str.extract(r"S(\d+)x")

fig, axes = plt.subplots(3, 3, figsize=(13, 10), sharey=False)
pdks_sorted = sorted(df_cross["pdk"].unique())
dims = [
    ("n_layers",     "Number of layers",    [2, 3, 5]),
    ("width_mult",   "Width multiplier",    [1, 2, 3]),
    ("spacing_mult", "Spacing multiplier", [1, 2, 3]),
]

for row_idx, (col, xlabel, xticks) in enumerate(dims):
    for col_idx, pdk in enumerate(pdks_sorted):
        ax = axes[row_idx][col_idx]
        subset = df_cross[df_cross["pdk"] == pdk]
        for tool in TOOLS:
            ts = subset[subset["tool"] == tool]
            if ts.empty:
                continue
            grouped = ts.groupby(col)["total_coupling_fF"].mean()
            ax.plot(grouped.index, grouped.values, "o-", label=tool,
                    color=tool_colors[tool], markersize=5)
        ax.set_xlabel(xlabel)
        ax.set_ylabel("Total coupling cap (fF)")
        ax.set_title(f"{pdk}")
        ax.legend(fontsize=7)
        ax.set_xticks(xticks)
        ax.set_yscale("log")

fig.suptitle("Cross Structures: Total Coupling Capacitance vs Sweep P
fig.tight_layout()
plt.show()
```



In [35]:

```
# Relative error summary: kpex vs magic across all structures
valid = cap_pivot[["kpex", "magic"]].dropna()
valid = valid[valid["magic"] > 0]
rel_err = (valid["kpex"] - valid["magic"]) / valid["magic"] * 100
print("kpex vs MAGIC — all structures:")
print(f"  Mean: {rel_err.mean():.1f}%  Median: {rel_err.median():.1f}%"
      f"  Std: {rel_err.std():.1f}%  MAE: {rel_err.abs().mean():.1f}%")
print()

# Breakdown by structure type
for struct in sorted(cap_pivot["structure"].unique()):
    sub = cap_pivot[cap_pivot["structure"] == struct][["kpex", "magic"]].
    sub = sub[sub["magic"] > 0]
    if sub.empty:
        continue
    re = (sub["kpex"] - sub["magic"]) / sub["magic"] * 100
    print(f"  {struct:10s}  mean={re.mean():+6.1f}%  median={re.median():
          f"  MAE={re.abs().mean():5.1f}%  n={len(sub)}")
```

```
kpex vs MAGIC — all structures:
  Mean: -15.3%  Median: -0.0%  Std: 33.5%  MAE: 17.9%

  cross       mean= -83.7%  median= -84.4%  MAE= 83.7%  n=144
  fringe      mean=  +3.8%  median=  +3.3%  MAE=  3.8%  n=216
  overlap     mean=  -3.1%  median=  -0.0%  MAE=  3.1%  n=27
  sidewall    mean=  -1.1%  median=  -0.0%  MAE=  2.0%  n=378
```

6. Error Analysis by Capacitance Type

Statistical comparison of kpex vs MAGIC across the three fundamental capacitance mechanisms. MAGIC is the reference. Relative error: `(kpex − magic) / magic × 100%`.

In [36]:

```
def pivot_type_df(df_type, type_name):
    piv = df_type.pivot_table(
        index=["cell", "pdk", "params"],
        columns="tool",
        values="total_coupling_fF",
    ).reset_index()
    piv["cap_type"] = type_name
    return piv


piv_overlap = pivot_type_df(df_overlap, "overlap")
piv_sidewall = pivot_type_df(df_sidewall, "sidewall")
piv_fringe = pivot_type_df(df_fringe, "fringe")

piv_by_type = pd.concat([piv_overlap, piv_sidewall, piv_fringe], ignore_i

err_rows = []
for _, row in piv_by_type.iterrows():
    ref_val = row.get("magic")
    kpex_val = row.get("kpex")
    if pd.isna(ref_val) or ref_val <= 0 or pd.isna(kpex_val):
        continue
    rel_err = (kpex_val - ref_val) / ref_val * 100
    err_rows.append({
        "cell": row["cell"],
        "pdk": row["pdk"],
        "params": row["params"],
        "cap_type": row["cap_type"],
        "kpex_fF": kpex_val,
        "magic_fF": ref_val,
        "rel_error_%": rel_err,
        "abs_error_fF": kpex_val - ref_val,
    })

df_err = pd.DataFrame(err_rows)
print(f"Error analysis rows: {len(df_err)}")
df_err.head()
```

```
Error analysis rows: 621
```

Out[36]:

	cell	pdk	params	cap_type	kpex_fF	magic_fF	rel_err
0	overlap_gf180_L2_W1x.gds	gf180	L2_W1x	overlap	0.165276	0.165276	-0.0
1	overlap_gf180_L2_W2x.gds	gf180	L2_W2x	overlap	0.330551	0.330551	0.0
2	overlap_gf180_L2_W3x.gds	gf180	L2_W3x	overlap	0.495827	0.495827	-0.0
3	overlap_gf180_L3_W1x.gds	gf180	L3_W1x	overlap	0.330551	0.330552	-0.0
4	overlap_gf180_L3_W2x.gds	gf180	L3_W2x	overlap	0.661102	0.661102	0.0

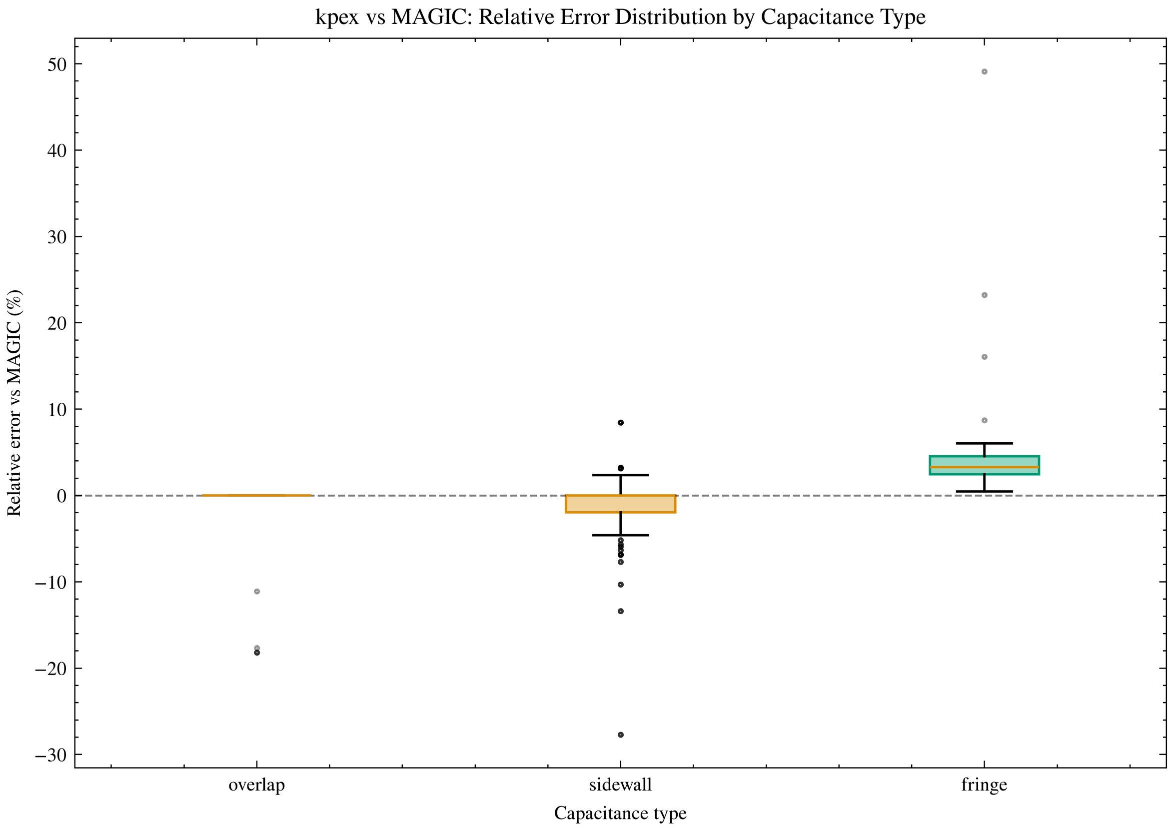
In [37]:
```python
# Summary statistics table
err_summary = df_err.groupby("cap_type")["rel_error_%"].agg(
    cells="count",
    mean="mean",
    median="median",
    std="std",
    min="min",
    max="max",
    MAE=lambda x: x.abs().mean(),
    RMSE=lambda x: np.sqrt((x**2).mean()),
).round(1)
err_summary.columns = ["cells", "mean (%)", "median (%)", "std (%)",
                        "min (%)", "max (%)", "MAE (%)", "RMSE (%)"]
print("kpex vs MAGIC — relative error by capacitance type:")
err_summary
```

kpex vs MAGIC — relative error by capacitance type:

Out[37]:

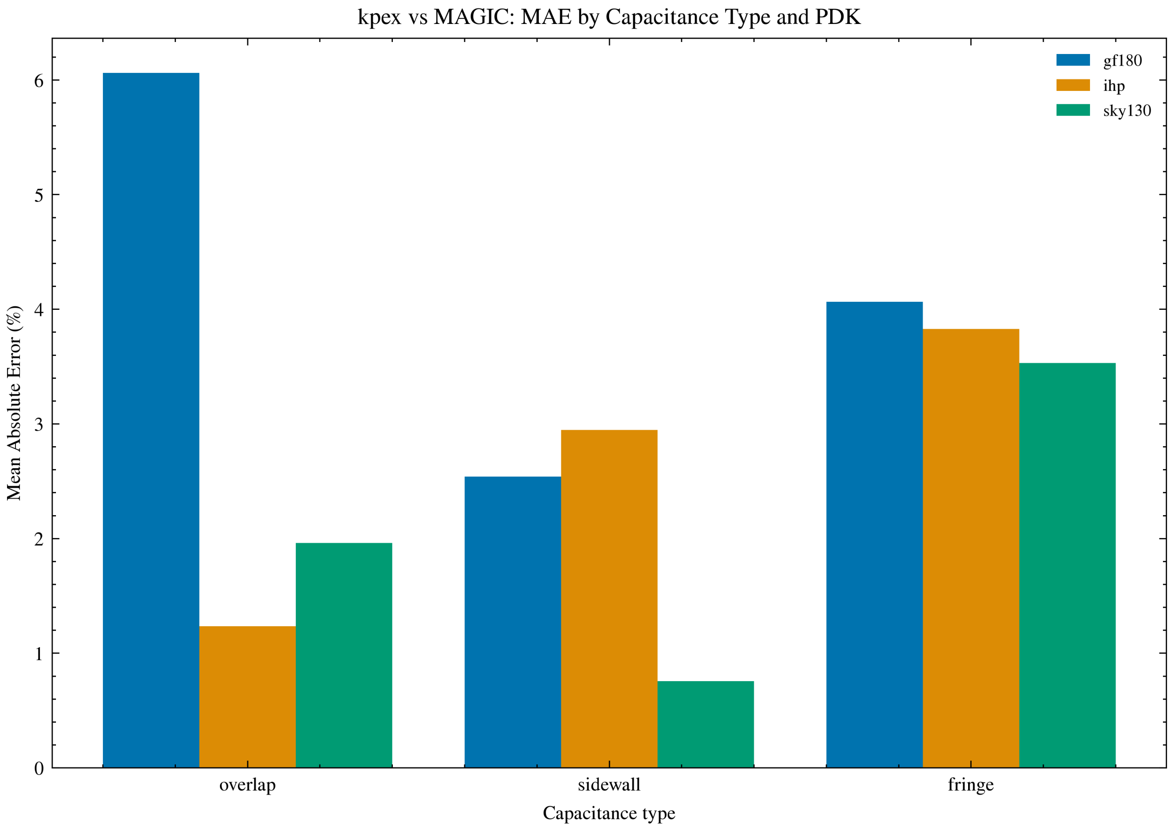
cap_type	cells	mean (%)	median (%)	std (%)	min (%)	max (%)	MAE (%)	RMSE (%)
fringe	216	3.8	3.3	3.7	0.5	49.1	3.8	5.3
overlap	27	-3.1	-0.0	6.7	-18.2	0.0	3.1	7.3
sidewall	378	-1.1	-0.0	3.8	-27.7	8.4	2.0	3.9

In [38]:
```python
# Box plot: relative error distribution by capacitance type
cap_types = ["overlap", "sidewall", "fringe"]
fig, ax = plt.subplots(figsize=(7, 5))

data = [df_err[df_err["cap_type"] == ct]["rel_error_%"] for ct in cap_typ
bp = ax.boxplot(data, tick_labels=cap_types, patch_artist=True, showflier
                flierprops=dict(marker=".", markersize=3, alpha=0.4))
colors = sns.color_palette("colorblind", n_colors=3)
for patch, color in zip(bp["boxes"], colors):
    patch.set_facecolor((*color, 0.4))
    patch.set_edgecolor(color)
ax.axhline(0, color="k", ls="--", lw=0.8, alpha=0.5)
ax.set_ylabel("Relative error vs MAGIC (%)")
ax.set_xlabel("Capacitance type")
ax.set_title("kpex vs MAGIC: Relative Error Distribution by Capacitance T
fig.tight_layout()
plt.show()
```



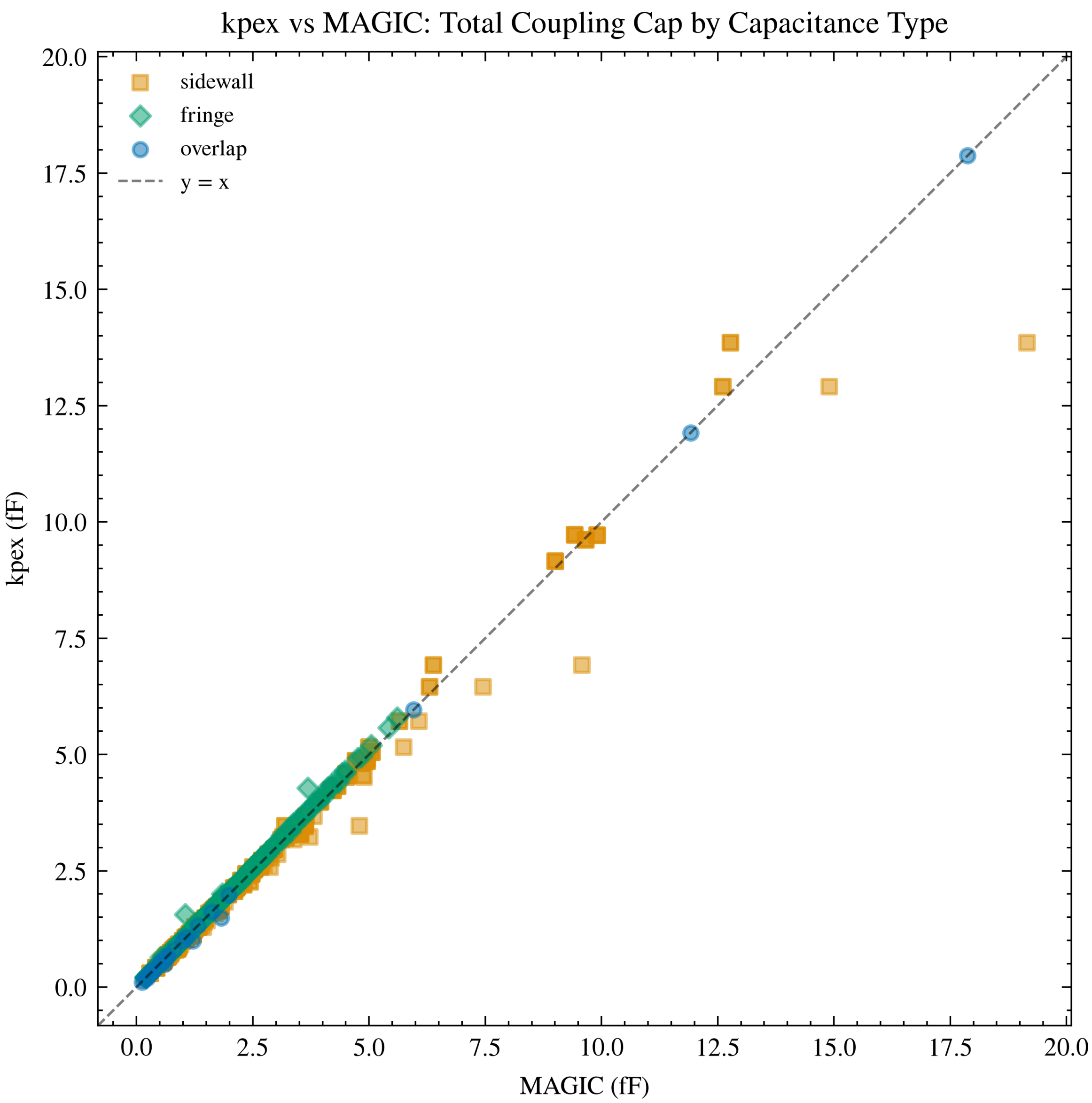
In [39]:

```
# MAE by capacitance type and PDK
mae_pdk = df_err.groupby(["cap_type", "pdk"])["rel_error_%"].apply(
    lambda x: x.abs().mean()
).reset_index(name="MAE (%)")

pdks = sorted(mae_pdk["pdk"].unique())
x = np.arange(len(cap_types))
width = 0.8 / len(pdks)
colors = sns.color_palette("colorblind", n_colors=len(pdks))

fig, ax = plt.subplots(figsize=(7, 5))
for i, pdk in enumerate(pdks):
    vals = [mae_pdk[(mae_pdk["pdk"] == pdk) & (mae_pdk["cap_type"] == ct)]
            for ct in cap_types]
    vals = [v[0] if len(v) else 0 for v in vals]
    ax.bar(x + i * width, vals, width, label=pdk, color=colors[i])
ax.set_xticks(x + width * (len(pdks) - 1) / 2)
ax.set_xticklabels(cap_types)
ax.set_ylabel("Mean Absolute Error (%)")
ax.set_xlabel("Capacitance type")
ax.set_title("kpex vs MAGIC: MAE by Capacitance Type and PDK")
ax.legend(fontsize=7)
fig.tight_layout()
plt.show()
```



In [40]:

```
# Scatter: kpex vs MAGIC coloured by capacitance type
cap_type_markers = {"sidewall": "s", "fringe": "D", "overlap": "o"}
cap_type_colors = {ct: c for ct, c in zip(cap_types, sns.color_palette("c

fig, ax = plt.subplots(figsize=(6, 5))
for ct, marker in cap_type_markers.items():
    subset = df_err[df_err["cap_type"] == ct]
    if subset.empty:
        continue
    ax.scatter(subset["magic_fF"], subset["kpex_fF"],
               marker=marker, alpha=0.5, s=20, label=ct,
               color=cap_type_colors[ct])

lims = [min(ax.get_xlim()[0], ax.get_ylim()[0]),
        max(ax.get_xlim()[1], ax.get_ylim()[1])]
ax.plot(lims, lims, "k--", lw=0.8, alpha=0.5, label="y = x")
ax.set_xlim(lims)
ax.set_ylim(lims)
ax.set_xlabel("MAGIC (fF)")
ax.set_ylabel("kpex (fF)")
ax.set_title("kpex vs MAGIC: Total Coupling Cap by Capacitance Type")
ax.set_aspect("equal")
ax.legend(fontsize=7)
fig.tight_layout()
plt.show()
```



In [41]:
```
# Per-PDK error breakdown
err_pdk = df_err.groupby(["cap_type", "pdk"])["rel_error_%"].agg(
    cells="count",
    mean="mean",
    median="median",
    std="std",
    MAE=lambda x: x.abs().mean(),
    RMSE=lambda x: np.sqrt((x**2).mean()),
).round(1)
err_pdk.columns = ["cells", "mean (%)", "median (%)", "std (%)", "MAE (%)
print("kpex vs MAGIC — relative error by capacitance type and PDK:")
err_pdk
```

```
kpex vs MAGIC — relative error by capacitance type and PDK:
```

Out[41]:

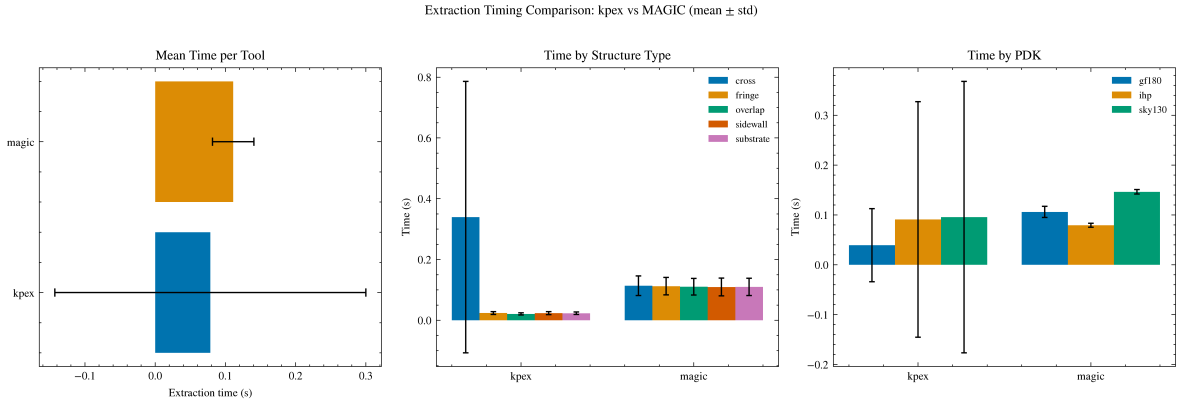
cap_type	pdk	cells	mean (%)	median (%)	std (%)	MAE (%)	RMSE (%)
fringe	gf180	54	4.1	4.2	1.3	4.1	4.3
	ihp	81	3.8	3.8	1.0	3.8	4.0
	sky130	81	3.5	2.6	5.9	3.5	6.9
overlap	gf180	9	-6.1	-0.0	9.1	6.1	10.5
	ihp	9	-1.2	-0.0	3.7	1.2	3.7
	sky130	9	-2.0	0.0	5.9	2.0	5.9
sidewall	gf180	108	-2.1	-1.9	2.4	2.5	3.2
	ihp	135	-0.6	-0.2	5.5	2.9	5.5
	sky130	135	-0.8	-0.0	2.0	0.8	2.1

7. Timing Comparison

In [42]:

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
tools_list = ["kpex", "magic"]

# Mean time per tool
ax = axes[0]
time_stats = df.groupby("tool")["time_s"].agg(["mean", "std"]).sort_value
colors = sns.color_palette("colorblind", n_colors=len(time_stats))
ax.barh(time_stats.index, time_stats["mean"], xerr=time_stats["std"], col
ax.set_xlabel("Extraction time (s)")
ax.set_title("Mean Time per Tool")

# Time by structure type
ax = axes[1]
time_by_struct = df.groupby(["tool", "structure"])["time_s"].agg(["mean",
structures = sorted(df["structure"].unique())
x = np.arange(len(tools_list))
width = 0.8 / len(structures)
for i, struct in enumerate(structures):
    s = time_by_struct[time_by_struct["structure"] == struct]
    vals = [s[s["tool"] == t]["mean"].values[0] if len(s[s["tool"] == t])
    errs = [s[s["tool"] == t]["std"].values[0] if len(s[s["tool"] == t])
    ax.bar(x + i * width, vals, width, yerr=errs, label=struct, capsize=2
ax.set_xticks(x + width * (len(structures) - 1) / 2)
ax.set_xticklabels(tools_list)
ax.set_ylabel("Time (s)")
ax.set_title("Time by Structure Type")
ax.legend(fontsize=7)

# Time by PDK
ax = axes[2]
time_by_pdk = df.groupby(["tool", "pdk"])["time_s"].agg(["mean", "std"]).
pdks = sorted(df["pdk"].unique())
width = 0.8 / len(pdks)
for i, pdk in enumerate(pdks):
    s = time_by_pdk[time_by_pdk["pdk"] == pdk]
    vals = [s[s["tool"] == t]["mean"].values[0] if len(s[s["tool"] == t])
```

```
    errs = [s[s["tool"] == t]["std"].values[0] if len(s[s["tool"] == t])
    ax.bar(x + i * width, vals, width, yerr=errs, label=pdk, capsize=2)
ax.set_xticks(x + width * (len(pdks) - 1) / 2)
ax.set_xticklabels(tools_list)
ax.set_ylabel("Time (s)")
ax.set_title("Time by PDK")
ax.legend(fontsize=7)

fig.suptitle("Extraction Timing Comparison: kpex vs MAGIC (mean ± std)",
fig.tight_layout()
plt.show()
```



In [43]:
```
# Timing summary statistics
time_summary = df.groupby("tool")["time_s"].agg(["count", "mean", "median
time_summary.columns = ["cells", "mean (s)", "median (s)", "std (s)", "mi
time_summary
```

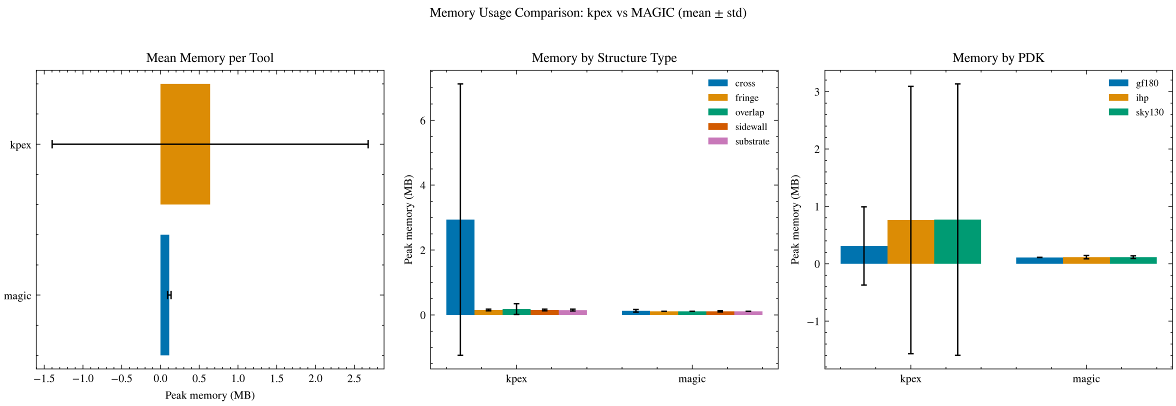
Out[43]:

tool	cells	mean (s)	median (s)	std (s)	min (s)	max (s)
kpex	825	0.078555	0.0255	0.221517	0.0144	1.9555
magic	825	0.111035	0.1051	0.029528	0.0740	0.2601

8. Memory Usage Comparison

In [44]:
```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

# Mean memory per tool
ax = axes[0]
mem_stats = df.groupby("tool")["memory_MB"].agg(["mean", "std"]).sort_val
colors = sns.color_palette("colorblind", n_colors=len(mem_stats))
ax.barh(mem_stats.index, mem_stats["mean"], xerr=mem_stats["std"], color=
ax.set_xlabel("Peak memory (MB)")
ax.set_title("Mean Memory per Tool")

# Memory by structure type
ax = axes[1]
mem_by_struct = df.groupby(["tool", "structure"])["memory_MB"].agg(["mean
x = np.arange(len(tools_list))
width = 0.8 / len(structures)
for i, struct in enumerate(structures):
    s = mem_by_struct[mem_by_struct["structure"] == struct]
    vals = [s[s["tool"] == t]["mean"].values[0] if len(s[s["tool"] == t])
    errs = [s[s["tool"] == t]["std"].values[0] if len(s[s["tool"] == t])
```

```
    ax.bar(x + i * width, vals, width, yerr=errs, label=struct, capsize=2
ax.set_xticks(x + width * (len(structures) - 1) / 2)
ax.set_xticklabels(tools_list)
ax.set_ylabel("Peak memory (MB)")
ax.set_title("Memory by Structure Type")
ax.legend(fontsize=7)

# Memory by PDK
ax = axes[2]
mem_by_pdk = df.groupby(["tool", "pdk"])["memory_MB"].agg(["mean", "std"]
width = 0.8 / len(pdks)
for i, pdk in enumerate(pdks):
    s = mem_by_pdk[mem_by_pdk["pdk"] == pdk]
    vals = [s[s["tool"] == t]["mean"].values[0] if len(s[s["tool"] == t])
    errs = [s[s["tool"] == t]["std"].values[0] if len(s[s["tool"] == t])
    ax.bar(x + i * width, vals, width, yerr=errs, label=pdk, capsize=2)
ax.set_xticks(x + width * (len(pdks) - 1) / 2)
ax.set_xticklabels(tools_list)
ax.set_ylabel("Peak memory (MB)")
ax.set_title("Memory by PDK")
ax.legend(fontsize=7)

fig.suptitle("Memory Usage Comparison: kpex vs MAGIC (mean ± std)", y=1.0
fig.tight_layout()
plt.show()
```



In [45]:

```
# Memory summary statistics
mem_summary = df.groupby("tool")["memory_MB"].agg(["count", "mean", "medi
mem_summary.columns = ["cells", "mean (MB)", "median (MB)", "std (MB)", "
mem_summary
```

Out[45]:

tool	cells	mean (MB)	median (MB)	std (MB)	min (MB)	max (MB)
kpex	825	0.640230	0.17	2.038611	0.11	14.51
magic	825	0.113261	0.11	0.022805	0.11	0.51

In [46]:

```
# Time vs memory scatter
fig, ax = plt.subplots(figsize=(6, 4))
for tool in TOOLS:
    subset = df[df["tool"] == tool]
    ax.scatter(subset["time_s"], subset["memory_MB"],
               alpha=0.5, s=15, label=tool, color=tool_colors[tool])
ax.set_xlabel("Extraction time (s)")
ax.set_ylabel("Peak memory (MB)")
```

```
ax.set_title("Time vs Memory: kpex vs MAGIC")
ax.set_xscale("log")
ax.set_yscale("log")
ax.legend()
fig.tight_layout()
plt.show()
```